

Designing biochemical circuits with tree search

Reviewed Preprint

v1 • June 24, 2025

Not revised

Pranav S Bhamidipati , Matthew Thomson 

Division of Biology and Biological Engineering, California Institute of Technology, Pasadena, United States • Keck School of Medicine, University of Southern California, Los Angeles, United States • Department of Computing and Mathematical Sciences, California Institute of Technology, Pasadena, United States • Beckman Center for Single-cell Profiling and Engineering, Pasadena, United States

 https://en.wikipedia.org/wiki/Open_access Copyright information

eLife Assessment

This manuscript presents a **valuable** computational tool for identifying 3-5 gene regulatory network topologies capable of generating oscillatory dynamics. The application of Monte Carlo Tree Search to circuit design is novel and effectively expands the scale at which non-linear behaviours can be explored in silico. The efficiency of the proposed algorithm is **convincing**, and the work will be of interest to the systems and synthetic biology communities. While the evolutionary implications remain unclear, the methodological contribution represents a significant advance in the field.

<https://doi.org/10.7554/eLife.106497.1.sa1>

Abstract

Discovering biochemical circuits that exhibit a desired behavior is an outstanding problem in biological engineering. The traditional approach of enumerating every possible circuit topology becomes intractable for circuits with more than four components due to combinatorial scaling of the search space. Here, we use Monte Carlo Tree Search (MCTS), a reinforcement learning (RL) algorithm, to optimize circuit topology for a target phenotype by approaching circuit design as a sequence of assembly decisions. Our RL-based design framework, which we call CircuiTree, efficiently and comprehensively finds robust designs for three-component oscillators by prioritizing sparsity. CircuiTree can also infer candidate network motifs from its search results, producing similar results to enumeration. Using parallel MCTS, we scale this workflow up to five components and find that highly fault-tolerant designs use a novel strategy, which we call motif multiplexing. Multiplexed circuits contain many overlapping network motifs that each activate in different mutational scenarios. The evolutionary robustness of multiplexing may explain the ubiquity of multiple sub-oscillators in circadian clock circuits. Overall, CircuiTree provides the first scalable computational platform for designing biochemical circuits.

Introduction

Understanding how biochemical networks produce biological behavior is one of the central goals of systems and synthetic biology. Studying the design principles of these networks, or circuits, provides fundamental insights into their computational capabilities and enables the application of synthetic biology to problems such as cell-based therapeutics (Lim et al. (2013) [↗](#); Williams et al. (2020) [↗](#)) and engineered microbial communities (Jones et al. (2024) [↗](#); Purnick and Weiss (2009) [↗](#)). The topology, or qualitative architecture, of a synthetic gene circuit is a critical determinant of its behavior, and choosing a topology is the first step in engineering circuits with novel functions such as switching in response to an input (Gardner et al. (2000) [↗](#)), spontaneous oscillations (Elowitz and Leibler (2000) [↗](#)), or multistability (Zhu et al. (2022) [↗](#)). Circuit topologies with a particular desired phenotype have traditionally been discovered *de novo* using a method we call computational enumeration. Enumeration involves listing every topology in a predefined space of possibilities and simulating its behavior with an exhaustive set of random perturbations of the modeling parameters (Ma et al. (2009) [↗](#); Cotterell and Sharpe (2010) [↗](#); Chau et al. (2012) [↗](#); Schaefer et al. (2014) [↗](#); Gerardin et al. (2019) [↗](#)). This method identifies the topologies that are the most robust to parameter choice, as well as the network motifs of the phenotype, which are topological patterns overrepresented among highly robust circuits (Alon (2007) [↗](#), 2019)).

Currently, computational complexity has limited the role of automation in circuit design. Assuming 3 possible types of interactions (activation, inhibition, or no interaction), the number of distinct topologies that can be made with N circuit components scales as $O(3^{N^2})$, not accounting for symmetries. Because the enumeration method requires simulating each topology in this potential space with many parameter perturbations (typically $\geq 10^4$), designing a five-component circuit with this approach would require on the order of 10^{15} - 10^{16} simulations, a computationally intractable number. This “curse of dimensionality” constitutes the main barrier to understanding the structure-function relationship for large circuits. The search space can be narrowed by only considering combinations of smaller motifs (Qiao et al. (2019) [↗](#)), but this approach risks excluding novel, non-modular designs that appear at higher complexity (Jiménez et al. (2017) [↗](#)). For large circuits, a functional topology may be found with evolutionary optimization (François et al. (2007) [↗](#); François and Siggia (2010) [↗](#)) or recurrent neural network-guided inference (Shen et al. (2021) [↗](#)), but these methods can be sensitive to the optimization parameters and the nature of the fitness landscape, a fact that complicates inference of network motifs and other general design principles. The ideal design framework for circuit topologies would be: (i) *Efficient*: Using relatively few samples, it returns a set of reasonably robust solutions. (ii) *Generalizable*: It generates solutions that are representative of the ground truth, enabling inference of motifs and design principles. (iii) *Scalable*: It maintains features (i) and (ii) when searching large spaces of potential designs.

Our inspiration for a circuit design approach with these features comes from algorithms developed in modern artificial intelligence to solve decision problems, which share core computational similarities with genetic circuit design. In a decision problem, or game, one starts from an initial position (the root state) and makes a sequence of decisions ending in success or failure. Algorithms for optimal game playing sample the branching tree of possible decisions in search of the best decision sequences, or paths. Large game trees, characteristic of long, complex games like chess ($\sim 10^{43}$ states) and Go (10^{170} states), can overwhelm classical exhaustive search methods (e.g., flat search, A^* , and minimax) and branch-and-bound methods (e.g., α - β pruning) (Shannon (1950) [↗](#); Tromp and Farnebäck (2007)). In contrast, most modern probabilistic search algorithms use reinforcement learning (RL) to sample decision paths; for example, Monte Carlo tree search (MCTS) uses an RL decision rule that optimistically estimates the success of each

possible decision (Kocsis and Szepesvári (2006 [↗](#))). MCTS has proven to be instrumental in solving large games (Silver et al. (2016 [↗](#), 2018)) due to its low memory footprint, ability to be parallelized (Enzenberger and Müller (2010 [↗](#))), balance of exploitation and exploration (Auer et al. (2002 [↗](#))), and myriad variations (Świechowski. (2022 [↗](#)); Browne et al. (2012 [↗](#))).

In this work, we present a circuit design platform based on tree search. This platform, which we call CircuiTree, approaches circuit design as a sequence of assembly decisions and optimizes the circuit topology for a given phenotype by using MCTS to search the tree of possible circuit assemblies. We first used CircuiTree to search a trial space – all 3,325 connected circuits with three transcription factors – for topologies that exhibit spontaneous, sustained oscillations. As it dynamically explores the search space, CircuiTree discovers sparse oscillators before complex ones and reliably identifies the most robust topologies. Next, we demonstrate that CircuiTree accurately identifies assembly motifs, which are overrepresented search states analogous to network motifs found by enumeration. The best assembly motifs for 3-component oscillators also contain the activator-inhibitor (AI) and/or repressor (Rep) motifs, a result consistent with previous studies of 2- and 3-component oscillators. Finally, we use a parallelized implementation of CircuiTree to study mutational tolerance in large circuits. We discover that highly fault-tolerant five-component oscillator circuits are “multiplexed” in that they contain multiple AI and Rep motifs that are syner-gistically interleaved such that the oscillations are robust to complete and partial knockdowns of single genes. Our results demonstrate that CircuiTree’s RL-based framework enables the efficient, generalizable, and scalable design of biochemical circuits, which could be used to engineer and understand complex intracellular and multicellular biological systems.

Results

Circuit design as an assembly game

We define a circuit as a system of k dynamically interacting biochemical species $\mathbf{x} = [x_0, \dots, x_k]$ characterized by its qualitative topology s , a member of the set of possible topologies $\mathcal{S}$, and its parameters $\theta \in \mathbb{R}^{n(s)}$, which describe the rates of biochemical reactions. We also assume the existence of a phenotype function $f(\mathbf{x}(t)) = q \in \{0, 1\}$ that returns a yes-or-no verdict as to whether a given simulation $\mathbf{x}(t)$ exhibits the desired phenotype. This function, which we conceptualize quite generally, may represent the presence of a dynamical phenotype such as adaptation or oscillation or a more general property such as multistability. Like many prior computational studies of design principles (Ma et al. (2009 [↗](#); Chau et al. (2012 [↗](#); Cotterell and Sharpe (2010 [↗](#); Schaefer et al. (2014 [↗](#))), each simulation of the system is performed with a set of randomly chosen parameters, and we define robustness $Q(s)$ as the mean phenotype score from these samples $Q(s) = \mathbb{E}_\theta[q(s)] = \mathbb{E}_\theta[f(\mathbf{x}(t; s, \theta))]$. The most robust topology s^* is therefore the solution to a combinatorial optimization problem. (Note that Q in this context should not be confused with the value function used in RL.)

$$s^* = \underset{s}{\operatorname{argmax}} Q(s) \quad (1)$$

In this work, for convenience, we consider models where all topologies are governed by the same reaction parameters (i.e. $n(s_i) = n(s_j) \forall s_i, s_j \in \mathcal{S}$), but CircuiTree does not require this assumption.

To solve Equation 1 [↗](#), we approach circuit topology design as a multi-step decision problem, or game (Figure 1A [↗](#)). Beginning with an “empty” circuit s_0 with a given set of components and without interactions, each step of the game can add an activating or inhibitory (auto-)regulatory interaction. The game ends when the builder chooses to “terminate” the assembly at some topology s_i , at which point the outcome is decided by a simulation with a win probability $Q(s_i)$. In the language of Markov decision processes (MDPs), each topology s represents a state of the game,

and the addition of an interaction or the termination of assembly represents an action a that converts a state s_i to a different state s_j . Each playout of the game traces a path on the tree of possible decisions T rooted at s_0 . The nodes in the tree are circuit topologies $\mathcal{R}$, and each directed edge $(s_i, s_j) \in \varepsilon$ represents the assembly of s_j from s_i by an action. Note that because multiple decision paths can converge on the same s_j , T is technically a directed acyclic graph (DAG), but we will call it a tree for simplicity.

For large design spaces, it is computationally infeasible to estimate $Q(s)$ for every $s \in T$. Instead, we iteratively sample from T using Monte Carlo tree search (MCTS), an RL algorithm commonly used in MDPs and game-playing artificial intelligences (Silver et al. (2016 [↗](#), 2017, 2018)) to bound the complexity of searching large decision trees. In these applications, tree search is used during game play (and sometimes also during training) to find the best move by simulating many hypothetical scenarios forward in time.

Each MCTS iteration starts at the root state s_0 and undergoes four phases, shown in **Figure 1B** [↗](#) and described in Algorithm 1. The key innovation of MCTS occurs during the first phase (selection), when it uses the UCT criterion from RL to choose moves while balancing (i) exploitation of moves that have yielded high rewards in the past and (ii) exploration of new moves that may yield higher rewards. At each branch (**Figure 1C** [↗](#), left), from a state s_i , multiple actions a_j are available that have yielded cumulative rewards r_{ij} over v_{ij} previous visits. MCTS decides the optimal action using the decision policy $a_i^* = \operatorname{argmax}_j \operatorname{UCT}(i, j) \operatorname{UCT}(i, j)$ (**Figure 1C** [↗](#), right), originally developed to solve the classic multi-armed bandit problem Auer et al. (2002 [↗](#)).

$$\operatorname{UCT}(i, j) = \frac{r_{ij}}{v_{ij}} + c \sqrt{\frac{\ln \sum_i v_{ij}}{v_j}} \quad (2)$$

The first term of the UCT criterion estimates the mean reward of the move $s_i \rightarrow s_j$, while the second term, derived from Hoeffding's inequality (Kocsis and Szepesvári (2006 [↗](#))), is an estimated upper bound on the sampling error. The second term ensures that a move with a historically low reward but relatively few samples ($v_j \ll \sum_i v_{ij}$) will still be tried occasionally to account for insufficient sample size. Thus, UCT is an optimistic predictor of a move's true mean reward. Note that the two terms encourage exploitation and exploration, respectively. (By definition, $\operatorname{UCT}(i, j) = +\infty$ if $s_i \rightarrow s_j$ has not been visited before).

Using the UCT criterion, actions are selected recursively, breaking any ties randomly, until the new state s_j is terminal (i.e. the assembly is finished) or has not been visited before during sampling. An unvisited s_j is added to T . Then, during the simulation phase, the outcome q of the trial is determined by taking random actions until the “terminate” action is chosen and simulating the resulting topology with randomly chosen reaction parameters and initial conditions. Finally, the history of rewards and visits is updated for each selected edge in T (backpropagation).

In addition to finding individual circuit topologies, an important goal of circuit design is to identify specific structural features, or network motifs, that lead to successful designs (Milo et al. (2002 [↗](#); Alon (2007 [↗](#); Ma et al. (2009 [↗](#); Cotterell and Sharpe (2010 [↗](#); Shah and Sarkar (2011 [↗](#); Chau et al. (2012 [↗](#); Lim et al. (2013 [↗](#); Schaerli et al. (2014 [↗](#))). Similarly, in our game-playing paradigm, a motif of the assembly game is a series of moves (an assembly *strategy*) that leads to topologies with a high rate of phenotypic success, on average. Thus, when a motif is present in the space of possible designs, it creates an enriched region in T , which can be exploited during the tree search to rapidly discover successful designs (**Figure 1D** [↗](#)).

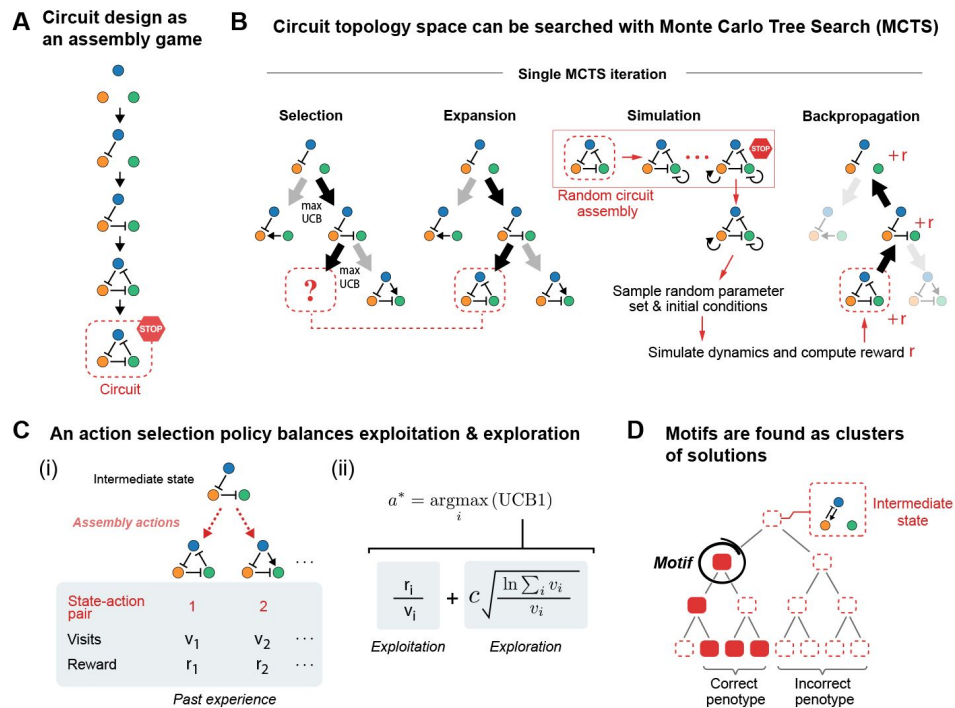


Figure 1.

A stepwise assembly framework enables circuit topology optimization with tree search.

(A) Circuit topologies are built step-by-step by adding interactions until the game is ended by taking the “terminate” action (the STOP sign). (B) Each MCTS iteration undergoes four phases: (1) Selection: The UCT criterion is used to recursively select the most promising action a_i^* from the current state s_i . (2) Expansion: If the edge (s_i, a_i^*) has not been sampled yet, it is added to the tree. (3) Simulation: If the circuit has not been completed yet, take random assembly moves until “terminate” is chosen. Simulate the resulting topology with random parameters and evaluate the reward q based on a target phenotype. (4) Backpropagation: update the history of rewards r_{ij} over past visits u_{ij} for the visited edges (s_i, a_i^*) . (C) The UCT policy for selecting a_i^* balances exploitation and exploration based on past trials. The exploration term, tuned by a hyperparameter c , favors actions that are under-sampled relative to the alternatives. (D) Once assembled, a motif creates an enriched subgraph in the search graph.

Establishing a ground-truth for three-node stochastic oscillators with enumeration

Oscillations appear in diverse cellular contexts such as metabolism (Chandra et al. (2011)), DNA damage (Geva-Zatorsky et al. (2006, 2010)), the cell cycle (Ferrell et al. (2011)), and circadian rhythms (Tyson et al. (1999)), and consequently, many of their design principles have been elucidated (Novák and Tyson (2008)). We first benchmark CircuiTree on the well-studied problem of designing an oscillator circuit with three nodes (Figure 2A, part i). Specifically, we consider a system of symmetric transcription factors (TFs) modeled as a stochastic birth-and-death process of individual mRNAs, TFs, and TF-response element (TF-RE) complexes, with elementary reactions of transcription, translation, degradation, binding, and unbinding (Figure S1A; reactions and their rate parameters are summarized in Table 1). TFs regulate transcription by binding to one of two REs in the regulatory region of each promoter, and cooperativity is introduced by assuming that TF-RE binding is stronger when both sites are occupied by the same TF. This assumption reduces the computational cost associated with modeling every homo- and heterodimer and their dimer-RE complexes. For each stochastic simulation, the system was randomly initialized and stochastically simulated using the Gillespie method for $t_{\max} = 4 \times 10^4 \text{ sec} = 11.1 \text{ hrs}$ (see Methods for details of initialization). Oscillations were quantified by computing the normalized autocorrelation function (ACF) and identifying its lowest minimum value across all TFs $\text{ACF}_{\min}$, excluding the bounds, and a Boolean reward value was assigned based on a cutoff value (Figure 2A, part ii).

$$r = \begin{cases} 1 & \text{if } \text{ACF}_{\min} < -0.4 \\ 0 & \text{if } \text{ACF}_{\min} > -0.4 \end{cases} \quad (3)$$

To compare with enumeration, all 3,325 unique, fully connected topologies were simulated with 10^4 randomly sampled parameter sets and initial conditions. The 10 rate parameters of the model were reduced to 8 dimensionless variables, which were sampled uniformly from a range of values determined based on experimentally measured rates (Milo et al. (2010)) and known requirements for oscillations (Elowitz and Leibler (2000); Novák and Tyson (2008)). (See Table 2 for variable definitions and sampling ranges and Methods for details of parameter sampling). The robustness to parameter perturbation Q was calculated as the fraction of parameter sets that produced oscillation, and topologies with $Q > 0.01$ were considered oscillators. This procedure uncovered 221 oscillator topologies (6.65% of the search space) (see Figures S1B and S1C for a summary of the best topologies). These results generally agree with prior studies of two- and three-node oscillators (Qiao et al. (2022); Elowitz and Leibler (2000); Novák and Tyson (2008); Woods et al. (2016); Stricker et al. (2008)). For instance, almost all oscillators contain a repressilator (Rep) loop, activator-inhibitor (A-I) loop, or a combination thereof. Positive autoregulation (PAR), which has been shown to buffer extrinsic and intrinsic noise (Qiao et al. (2022)), is also ubiquitous among these oscillators and is required for robust A-I loop oscillations (Novák and Tyson (2008); Stricker et al. (2008)). Further discussion of these motifs can be found in the section below. Interestingly, stochastic switching between stable states was occasionally mistaken for low-frequency oscillations (Figure S2), causing some toggle switch circuits to be classified as oscillators (highest $Q = 0.024$, ranked #139).

Reaction	Rate parameter	Value in Fig. 4D & Fig. 5
TF-RE binding	k_{on}	1.0 sec^{-1}
TF-RE unbinding (uncooperative)	$k_{\text{off},1}$	$99.0 \text{ molec}^{-1}\text{sec}^{-1}$
TF-RE unbinding (cooperative)	$k_{\text{off},2}$	$9.9 \text{ molec}^{-1}\text{sec}^{-1}$
Transcription, basal	k_{unbound}	0.05 sec^{-1}
Transcription, 1-2 activators	k_{act}	8.0 sec^{-1}
Transcription, 1-2 inhibitors	k_{inh}	$5 \times 10^{-4} \text{ sec}^{-1}$
Transcription, 1 activator and 1 inhibitor	k_{mixed}	0.05 sec^{-1}
Translation	k_p	$0.167 \text{ molec}^{-1}\text{sec}^{-1}$
Degradation, mRNA	γ_m	$0.025 \text{ molec}^{-1}\text{sec}^{-1}$
Degradation, protein	γ_p	$0.025 \text{ molec}^{-1}\text{sec}^{-1}$

Table 1.

Model rate parameters

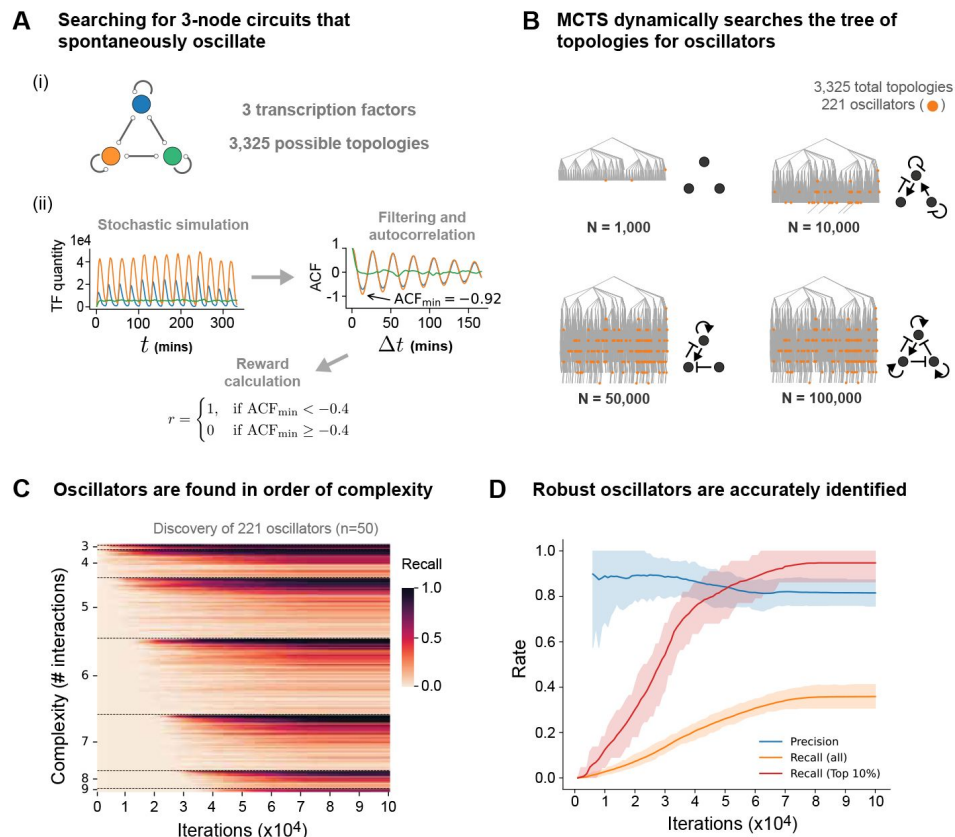


Figure 2.

CircuiTree efficiently identifies simple and robust 3-component oscillators.

(A) All three-component transcription factor (TF) circuits (3,325 up to symmetry) were enumerated with 10^4 random parameter sets (i) and evaluated for oscillations (ii) using an autocorrelation-based reward function. (B) A representative MCTS run. With more iterations (N), the search graph T (represented by a spanning tree for simplicity) expands to encounter more oscillators (orange circles) and improve its best predicted oscillator topology (shown in black). (C) A heatmap showing the average rate of discovery, or recall, for each oscillator (proportion of $n = 50$ replicates). Rows (oscillators) are sorted in order of complexity, or the number of interactions, and oscillators with the same complexity are sorted by descending robustness Q . Sparse oscillators are found before more complex ones, with a preference for the most robust candidates. (D) Precision (blue) and recall (orange) of oscillator classification (mean $\pm$ 95% CI, $n = 50$). CircuiTree's recall is particularly high for the 10% most robust oscillators (red), reaching 94.7% after 10^5 iterations. See also Figures S1, S2, S3, and S4.

Dimensionless variable	Definition	Sampling limits
κ_1	$\log_{10} [K_{D,1} \cdot 1 \text{ molec}]$	$[-2, 4]$
κ_2	$K_{D,2}/K_{D,1}$	$[0.00, 0.25]$
k'_{unbound}	$k_{\text{unbound}} \cdot 1 \text{ sec}$	$[0, 1]$
k'_{act}	$k_{\text{act}} \cdot 1 \text{ sec}$	$[1, 10]$
r_{inh}	$\log_{10} [k_{\text{inh}}/k_{\text{act}}]$	$[0, 5]$
k'_p	$k_p \cdot 1 \text{ molec} \cdot 1 \text{ sec}$	$[0.015, 0.250]$
γ'_m	$\log_{10} [\gamma_m \cdot 1 \text{ molec} \cdot 1 \text{ sec}]$	$[-3, -1]$
γ'_p	$\log_{10} [\gamma_p \cdot 1 \text{ molec} \cdot 1 \text{ sec}]$	$[-3, -1]$

Table 2.

Dimensionless variables and limits imposed on random parameter sampling

Algorithm 1 Circuit topology search using MCTS

Algorithm 1 Circuit topology search using MCTS

procedure MonteCarloTreeSearch(s_0, N)

$T \leftarrow \text{EmptyTree}()$

AddNode(T, s_0)

▷ Initialize T with the root topology s_0

repeat

$[s_0 \cdots s_n] \leftarrow \text{Select}(T, s_0)$

▷ Choose assembly moves and *select* the last topology

if s_n not in Nodes(T) **then**

AddNode(T, s_n)

▷ *Expand* the search graph if necessary

end if

if (s_{n-1}, s_n) not in Edges(T) **then**

AddEdge($T, (s_{n-1}, s_n)$)

end if

$s_{\text{sim}} \leftarrow s_n$

▷ Make random moves until termination occurs

while IsNonterminal(s_{sim}) **do**

$s_{\text{sim}} \leftarrow \text{DoRandomMove}(s_{\text{sim}})$

end while

$q \leftarrow \text{Simulate}(s_{\text{sim}})$

▷ *Simulate* s_{sim} and obtain a reward q

for $i := 0$ to $n - 1$ **do**

▷ *Backpropagate* the results of each decision

$v[s_i, s_{i+1}] \leftarrow v[s_i, s_{i+1}] + 1$

$r[s_i, s_{i+1}] \leftarrow r[s_i, s_{i+1}] + q$

end for

until N iterations or Timeout

return T, v, r

end procedure

procedure Select(T, s_0)

$i \leftarrow 0$

while Nonterminal(s_i) **do**

$S' \leftarrow \text{PossibleMoves}(s_i)$

$s_{i+1} \leftarrow \underset{s_j \in S'}{\text{argmax}} \text{UCB}(s_i, s_j)$

▷ Choose the assembly move that maximizes the UCB score

if $v[s_i, s_{i+1}] == 0$ **then**

▷ Until a new move is encountered

return $[s_0 \cdots s_{i+1}]$

end if

$i \leftarrow i + 1$

end while

▷ Or termination occurs

return $[s_0 \cdots s_{i+1}]$

end procedure

Across all oscillators, oscillation was favored by tight TF-RE binding ($K_{D,1} < 10^3$), low basal transcription ($k_{m,\text{unbound}} < 0.3 \text{ sec}^{-1}$), strong repression ($k_{m,\text{rep}}/k_{m,\text{unbound}} < 10^{-1}$), and a high activated transcription rate ($k_{m,\text{act}}$, monotonic effect) (Figure S3A). Protein and mRNA degradation rates had a non-monotonic effect with a peak at $\gamma_m \approx \gamma_p \approx 0.2 \text{ molec}^{-1} \text{ sec}^{-1}$ (Figure S3A), and oscillation period depended strongly on these rates and their ratio (Figures S3B and S3C). However, the best oscillators were exceptionally robust to the choice of parameters. For example, the most robust oscillator (the repressilator with PAR) has a Q of 0.767, meaning that for each of the 8 sampled variables, on average, 96.7% of the possible values would permit oscillations ($0.967^8 \approx 0.767$).

CircuiTree efficiently and systematically discovers three-node oscillators

MCTS masters a game with a limited number of trials by balancing deep sampling of promising regions with broad sampling of under-explored regions. To understand how this strategy performs for circuit topology design, we use CircuiTree to search for 3-node oscillators given $N = 10^5$ iterations of MCTS ($n = 50$ replicates; see Methods for MCTS implementation specifics). In the average run, the first putative result is discovered after just 0.69 samples/topology (2,280 iterations), and by 19.6 samples/topology (65,360 iterations), a top-5 oscillator has been sampled >100 times. A topology is discovered as a “successful” oscillator if its estimated robustness $\hat{Q}_j = r_j/v_j$ exceeds the threshold value $Q_{\text{thresh}} = 0.01$, which was chosen heuristically prior to the search. **Figure 2B** shows a representative example of how, over sampling time, CircuiTree incrementally builds the search tree from the root state (top to bottom), encounters more oscillators (shown in orange), and improves its prediction for the most robust design (shown as a black circuit diagram). By the end of sampling time, the average run saturates the tree, sampling 99.97% of topologies at least once.

CircuiTree balances efficiency and comprehensiveness by first finding sparse solutions before exploring deeper areas of the tree. In **Figure 2C**, each row of the heatmap is one of the 221 oscillators, and the rows are sorted first by the circuit’s complexity, or the number of interactions, then by decreasing robustness (Q). The color scale indicates the likelihood of discovery, or recall, of each oscillator measured as the proportion of replicates that discovered it. Because sparse topologies require fewer assembly steps, MCTS encounters the sparsest oscillators (such as the repressilator) first, and it discovers increasingly complex solutions over time until oscillators with 9 interactions (the maximum) are found at $N \approx 5 \times 10^4$. As shown by the line plot in **Figure 2D**, CircuiTree has a very high recall of 94.7% for the top 10% of oscillators (95% CI: 86.4% – 100.0%) and has a recall of 35.9% (95% CI: 30.8% – 41.5%) for oscillators in general. This is fairly high, considering that it gets an average of ~ 30 samples/topology, and a majority of oscillators have a Q of less than 1/30. Additionally, CircuiTree has a precision of 81.5% (95% CI: 74.1% – 88.3%), indicating a low rate of false positives.

The competing goals of breadth and depth manifest during tree search as distinct temporal phases in which MCTS first explores a broad set of topologies before focusing on a narrow, enriched subset. Among topologies containing a combination of AI and Rep feedback loops, 27.2% (47/173) are oscillators, a very high proportion compared to 6.7% (221/3,325) for the entire design space and 9.3% (146/1,575) and 8.0% (6/75) for the AI or Rep loops alone (Figure S4A). Figure S4B shows how MCTS allocates samples to each of these categories over a total sampling time of 10^5 and 10^6 iterations (mean of $n = 50$ and $n = 12$ replicates, respectively). During an initial exploratory phase, samples are taken broadly across categories; however, at $N \approx 6 \times 10^4$, the AI-Rep combination becomes heavily favored, followed by Rep alone, for the rest of sampling time. This transition can be observed directly from the search history by measuring regret. Defined as

$$R_N = NQ^* - \sum_{n=1}^N r_n, \quad (4)$$

regret is the difference between the expected reward from sampling the best topology (with robustness Q^*) and the actual reward over N iterations. As MCTS moves from exploration into exploitation, the reward rate becomes higher and regret accumulates more slowly (Figures S4C and S4D). Thus, the discovery of an enriched region during search triggers a shift to more focused sampling that can be identified in real-time without *a priori* knowledge about design principles or motifs.

CircuiTree infers oscillator motifs from search results

In addition to finding individual circuit topologies, an important goal of circuit design is to identify specific structural features, or network motifs, that lead to successful designs (Milo et al. (2002) [↗](#); Alon (2007) [↗](#); Ma et al. (2009) [↗](#); Cotterell and Sharpe (2010) [↗](#); Shah and Sarkar (2011) [↗](#); Chau et al. (2012) [↗](#); Lim et al. (2013) [↗](#); Schaerli et al. (2014) [↗](#)). Similarly, in our game-playing paradigm, a motif of the assembly game is a series of moves (an assembly strategy) that leads to a subset of T highly enriched with successful design results. Specifically, we define an assembly motif as any successful topology ($Q > Q_{\text{thresh}}$) that is also overrepresented as a sub-pattern within other successful topologies, compared to a set of randomly assembled designs. Using a statistical test for overrepresentation (described in detail in the Methods section and shown schematically in Figure S5A), CircuiTree can mine the results of a tree search for putative assembly motifs.

Looking for 3-node oscillator motifs with ≤ 3 interactions, CircuiTree identifies the same four motifs as enumeration, shown at the top of **Figure 3** [↗](#): the repressilator motif (Rep) and the activator-inhibitor (AI) loop with either PAR of the activator (AIPAR), constitutive inhibition of the inhibitor (AICI), or constitutive activation of the activator (AICA). CircuiTree finds these minimal motifs in 100% (Rep), 86% (AIPAR), 94% (AICI), and 62% (AICA) of replicates. To see how these motifs are situated in the overall design space, we plot all 221 oscillators in **Figure 3** [↗](#) as a “complexity atlas” (Cotterell and Sharpe (2010) [↗](#); Schaerli et al. (2014) [↗](#)), a graph-of-circuits where every oscillator is a node and nodes are organized in layers according to their complexity. Edges connect nodes in adjacent layers if they differ by the addition of a single circuit interaction, analogous to a move in the assembly game. Notably, 216/221 oscillators constitute a large connected component of the graph originating from the four minimal motifs, indicating that almost all 3-node oscillators live in a subset of design space defined by Rep and/or AI motifs. The size of each node in **Figure 3** [↗](#) denotes its motif robustness Q_{motif} the overall oscillation probability for all circuits containing this motif (in other words, the mean reward once reaching this state in the game). The color of each node reflects its motif discovery rate, measured as the percentage of $n = 50$ replicates that labeled it as a motif.

Note that these features correlate visually (large circles tend to be red and *vice versa*) and quantitatively, as shown by the scatterplot in Figure S5B, indicating that higher quality motifs are more likely to be found. Additionally, the motifs that are discovered most reliably (discovery rate $< 80\%$, indicated by bold edges) form an optimal subset corresponding to the best motifs (the largest nodes). Surprisingly, the most robust oscillator, the repressilator with PAR on all three TFs (Rep+3xPAR, $Q = 0.767$), is discovered at a rate of 98%. Because Rep+2xPAR is a poor oscillator (ranked #187) and Rep+1xPAR does not oscillate at all, Rep+3xPAR cannot be assembled from Rep (or any other intermediate) without breaking oscillations. Nonetheless, its high motif discovery rate suggests that CircuiTree is capable of identifying special design strategies that require a specific combination of moves. Overall, we find that CircuiTree reliably infers the most robust motifs for a given phenotype, even finding obscure but optimal solutions, and the resulting motifs for 3-node oscillation form a single cluster of optimal designs. Note that, in contrast to classic network motifs which compare the frequency of a pattern in successful topologies against the entire set of topologies (or a comparable null model), we compare frequencies between circuit assemblies that were successful during search and random circuit assemblies. Therefore our method returns motifs that are overrepresented *among search results* and thereby accounts for the bias inherent to sampling the space of topologies by assembly rather than by flat enumeration.

Motif multiplexing allows five-node oscillators to compensate for deletions and single-gene knockdowns

While synthetic circuits generally rely on a single, minimal circuit module, naturally occurring circuits often use many functional modules. For instance, circadian oscillators across divergent taxa contain two or more oscillatory feedback loops (Lee et al. (2000) [↗](#); Cheng et al. (2001) [↗](#);

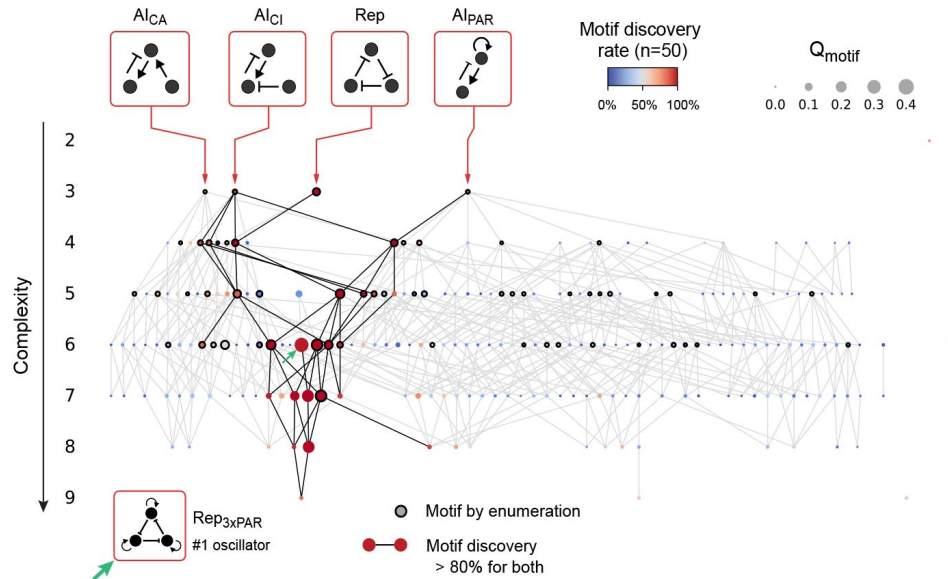


Figure 3.

Motifs identified from search results form a cluster of optimal 3-node oscillators.

A complexity atlas of oscillators with ≤ 3 components. Circles are oscillator topologies identified by enumeration, and edges link oscillators that differ by the addition/removal of one interaction. 97.7% of oscillators (216/221) are topologically related to one of the four motifs for 3-node oscillation, shown above the atlas in red boxes. Bold circle borders indicate oscillators found to be motifs based on enumeration. Circle color indicates the rate with which CircuiTree labels each oscillator as an assembly motif. Circle size indicates Q_{motif} , the average robustness for a circuit completed randomly starting from this state of the assembly game. The correlation between discovery rate and Q_{motif} (plotted in Figure S5B) suggests that motifs found by CircuiTree correspond to beneficial game states. The bolded edges, which connect oscillators with a discovery rate $> 80\%$, form a contiguous cluster representing optimal assembly strategies. The most robust oscillator, the repressilator with PAR of all components, is shown on the bottom-left and indicated on the atlas by a green arrow. See also Figures S4 and S5.

Bell-Pedersen et al. (2005) [\[1\]](#); Pokhilko et al. (2012) [\[2\]](#)). Could these additional modules make circadian oscillators more resistant to deletions during evolution (Wagner (2005) [\[3\]](#))? To explore this possibility, we implemented a parallelized version of CircuiTree with pruning (described in details in the Methods and illustrated in Figure S6) and used it to design 5-node circuit topologies that oscillate despite a 50% chance of deletion of a random TF (**Figure 4A** [\[4\]](#)). Please see Methods for a description of the parallel implementation and Figure S6 for a schematic of the pruning.

After 5 million search iterations (less than four days of real time using 1,000 parallel search threads and 300 CPUs), CircuiTree finds 1,386 putative oscillators with $\hat{Q}_i > 0.01$ and > 100 samples, the first result arriving in just 1.3 hours. Due to the computational cost of exact simulations of large stochastic systems, only topologies with up to 15 interactions were included. As shown by the heatmap in Figure S7A, when clustered based on their structural similarity (measured as the pairwise graph edit distance between topologies), these topologies separate into sparse and dense clusters (Figure S7B). In **Figure 4B** [\[5\]](#), each of these topologies is plotted as a circle based on their observed robustness with or without a random deletion ($\hat{Q}_{del}$ and $\hat{Q}_{model}$, respectively). The overall robustness $\hat{Q}$ is indicated by the color gradient, and the circle size represents the number of samples. The circuit's fault-tolerance (FT), the proportion of components that can be deleted without losing oscillation, is estimated as $\hat{Q}_{deletion}/\hat{Q}_{nodeletion}$, and contour lines for $FT = \frac{2}{3}, \frac{1}{2}$, and $\frac{1}{3}$ are shown as dashed lines. The putative oscillators included 42 topologies with 2 or 3 nodes, all of which had low robustness to deletions ($\hat{Q}_{deletion} < 0.32$). For instance, the second-best oscillator in the 3-node search, the AI+Rep circuit (labeled (i) in **Figure 4B** [\[6\]](#) and **Figure 4C** [\[7\]](#)), was found to have a fault tolerance of $0.404 \pm 0.009 \approx \frac{2}{5}$, consistent with oscillations that persist after deletion of the two unused TFs (D and E) but attenuating with deletion of any of the active components (A, B, or C).

In contrast, fault-tolerant oscillators ($\hat{Q}_{deletion} \gtrsim 0.4$) contain many interleaved oscillatory motifs that activate under different deletion scenarios, a design feature we call motif multiplexing. As shown in **Figure 4D** [\[8\]](#), the number of repressilator (Rep) and activator-inhibitor (AI) motifs is a strong predictor of robustness in the presence of deletion, measured as $\hat{Q}_{deletion}$, and higher complexity in general is associated with higher $\hat{Q}$ (Figure S7C). To see this design principle, consider the AI+3Rep circuit (labeled (ii) in **Figures 4B** [\[9\]](#) and **4C** [\[10\]](#)), which is an extension of the AI+Rep topology with two additional backup repressilator loops (A-C-D and A-C-E, highlighted in yellow). This circuit has a higher fault tolerance of $0.617 \pm 0.011 \approx \frac{3}{5}$ because, while deletion of gene B is fatal for oscillations in the AI+Rep circuit, the A-C-D motif takes over to rescue oscillations in the AI+3Rep circuit (representative simulated trajectories shown in **Figure 4E** [\[11\]](#), upper and middle row; see **Table 1** [\[12\]](#) for parameter values). The pattern extends to the 3AI+3PAR circuit (labeled (iii) in **Figures 4B** [\[13\]](#) and **4C** [\[14\]](#), right diagram) which is similar to the AI+3Rep except for the addition of two activator-inhibitor motifs (B-D and D-C, highlighted in yellow). This circuit similarly activates a repressilator motif (A-C-E) upon deletion of gene B. Now, however, oscillations are rescued after deletion of gene C by activating the B-D motif. Consequently, the 3AI+3PAR circuit has a high fault tolerance of $0.715 \pm 0.074 \approx \frac{4}{5}$ (**Figure 4E** [\[15\]](#), bottom row).

During evolution, genomic mutation may lead to a partial reduction in transcription rate rather than a complete knockout. Do multiplexed oscillators maintain their mutational robustness in these conditions? To explore this question, we simulated the 3AI+3Rep circuit under conditions where a single gene is partially knocked down by multiplying its transcription rate by a factor $(100 - KD)/100$, KD being the percent of knockdown. In **Figure 5A** [\[16\]](#), we visualize a representative stochastic simulation of the wild-type (WT) 3AI+3Rep circuit by reducing the system of five TFs (a five-dimensional phase space) to two composite dimensions using principal components analysis and plotting a simulated trajectory on the first and second principal component axes. At each time point, the dominantly expressed TF is indicated by a transparent marker of the same color, and the inset diagram shows the order of TFs activated in the limit cycle (A-B-D-C). As gene A is knocked down from KD=0% to KD=100% (**Figure 5B** [\[17\]](#), upper middle panel), the limit cycle

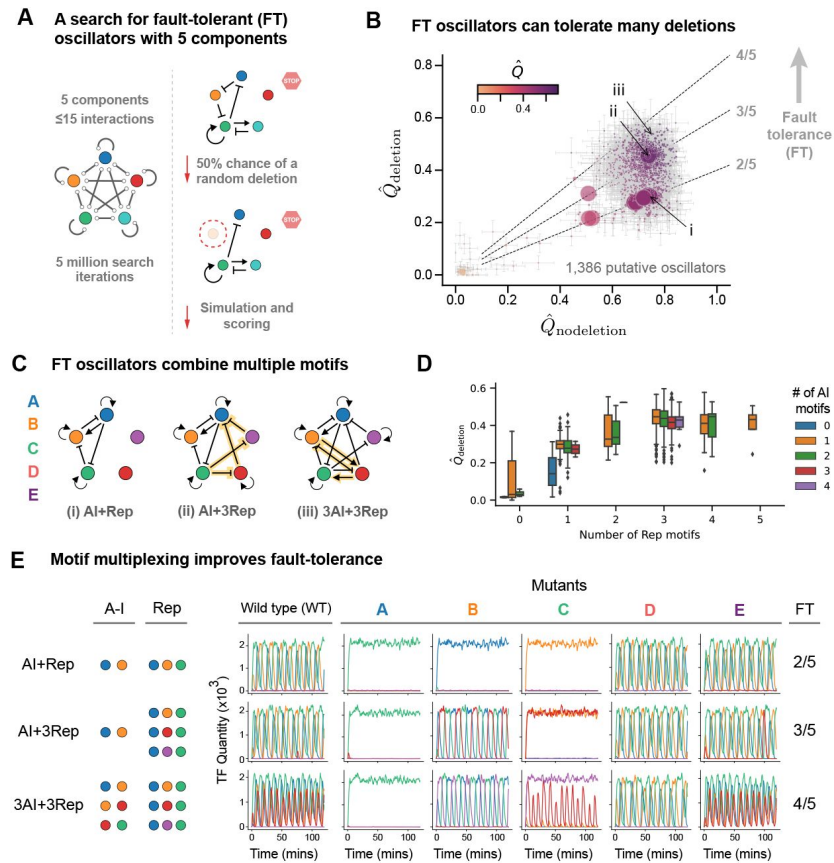


Figure 4.

Parallelized CircuiTree scales to large design spaces.

(A) A parallelized version of CircuiTree was used to search for five-node oscillators with ≤ 15 interactions (left) that oscillate despite a 50% chance of a single random deletion (right). (B) Search results after $5 \cdot 10^6$ iterations. Circles are putative oscillators ($\hat{Q} > 0.01$, $v_i > 100$), plotted on axes of robustness based on samples with ($\hat{Q}_{\text{deletion}}$) or without a deletion ($\hat{Q}_{\text{nodeletion}}$). Error bars indicate standard error of the mean. Circle size and color indicate the number of samples v_i and the overall robustness $\hat{Q}$, respectively. Dashed lines show different theoretical values of fault-tolerance (FT), or the average number of deletions a circuit can sustain. Examples of circuits with different FT (labeled i-iii) are shown in (C). (D) Box plots of $\hat{Q}_{\text{deletion}}$, grouped by the number of motifs in each putative oscillator. Multiple motifs, particularly Rep, increase robustness to deletions. (E) Simulated trajectories for the topologies in (C) for all single deletions. The 3AI+3Rep oscillator (FT $\approx 4/5$) contains 6 interleaved oscillator motifs. See also Figures S6 and S7.

gradual drifts in phase space until the trajectory eventually flattens, consistent with the lack of oscillations after deletion of gene A (**Figure 4E**). During knockdown of gene B (**Figure 5B**, top right panel), in contrast, the limit cycle drifts before discontinuously jumping to a new limit cycle (the A-E-C repressilator motif) between 75% and 100% KD, as shown in the inset diagram. Similarly, knockdown of gene C induces a gradual drift followed by a jump between 75% and 100% KD to a limit cycle driven by the B-D activator-inhibitor motif. Unlike genes B and C, knockdown of gene D produces no obvious discontinuities as the limit cycle gradually adapts from A-B-D-C to the A-B-C repressilator. Knockdown of gene E produces no appreciable changes in the limit cycle. Overall, the multiplexed oscillator 3AI+3Rep maintains oscillatory behavior during partial knockdown of genes B, C, D, or E and transitions between disparate limit cycles in qualitatively unique ways.

To quantitatively understand how the system responds to knockdowns, single-gene knock-downs were simulated with a range of values of KD with $n = 50$ replicates with different random seeds. The quality and frequency of oscillations were then assessed by computing the ACF_{min} and, if oscillations were detected, the frequency of oscillation. In **Figure 5C** (upper panel), the ACF_{min} (mean $\pm$ 95% confidence interval) is plotted as a function of KD, and the dashed line indicates the value $ACF_{min} = -0.4$ used as a threshold between oscillatory and non-oscillatory dynamics (below and above the line, respectively). Knockdown of gene A leads to a rise in ACF_{min} and becomes lethal for oscillations above $KD = 60\%$. For gene B, ACF_{min} increases before suddenly crossing the threshold to indicate loss of oscillations between $KD = 85\%$ and 95% , and for gene C, oscillations disappear in the range $KD = 65\%$ to 90% . For genes D and E, no detrimental effects are observed during knockdown. Thus, oscillations persist in almost all cases (91.9% of samples) when partially knocking down genes B, C, D, or E (**Figure 5C**, middle panel). Partial knockdowns of A, B, C, and D each have distinct effects on oscillation frequency, a phenomenon called frequency pulling (Heltberg et al. (2021)). The bottom panel of **Figure 5C** shows how, for $KD < 60\%$, the WT resonant frequency of 6.0hr^{-1} is pulled higher when knocking down A or B and lower when knocking down C or D (no change for E). At higher values of KD, knockdowns of B and C discontinuously jump from the drifted frequency to a new resonant frequency (2.6hr^{-1} and 6.6hr^{-1} , respectively), while the D knockdown transitions smoothly to its new frequency of 3.8hr^{-1} . Thus, the 3AI+3Rep oscillator accommodates most partial knockdowns of genes B, C, D, or E by modulating, yet retaining, oscillatory dynamics.

Up to this point, our modeling has assumed that the rates of reactions such as binding, transcription, translation, and degradation are identical between circuit components. To investigate whether the oscillators we discover are fine-tuned for this symmetry, we perturbed the rate parameters for each TF individually. A Gaussian perturbation kernel with standard deviation σ_{param} was applied to each parameter, scaled to the pre-defined ranges of those parameters (see **Table 2** for parameter ranges and Methods for details of the perturbation kernel). Stochastic simulations were performed for a range of values of σ_{param} between $1 \cdot 10^{-3}$ and $3 \cdot 10^{-2}$. To assess the effects of parameter perturbation on oscillation, the power spectral density (PSD) of the dynamics of TF A (mean of $n = 50$ replicates with different random seeds) was calculated for each value of σ_{param} and plotted as a heatmap (**Figure 5D**). For $\sigma_{param} < 10^{-2}$, the PSD shows peaks at the oscillator's fundamental frequency of 6hr^{-1} and first and second harmonics (12hr^{-1} and 18hr^{-1} , respectively). As σ_{param} increases further to $5 \cdot 10^{-2}$, the fundamental frequency peak gradually diffuses, and the oscillation rate drops from 1.0 to ~ 0.5 , as shown in the line plot in **Figure 5D** (envelope indicates 95% confidence interval). above $\sigma_{param} = 5 \cdot 10^{-2}$, the oscillation rate drops to near-zero, and the mean PSD shows no visible peaks. Therefore, this oscillator can tolerate a minimal amount of heterogeneity between TFs, above which it appears somewhat sensitive to asymmetry in at least one parameter of the model.

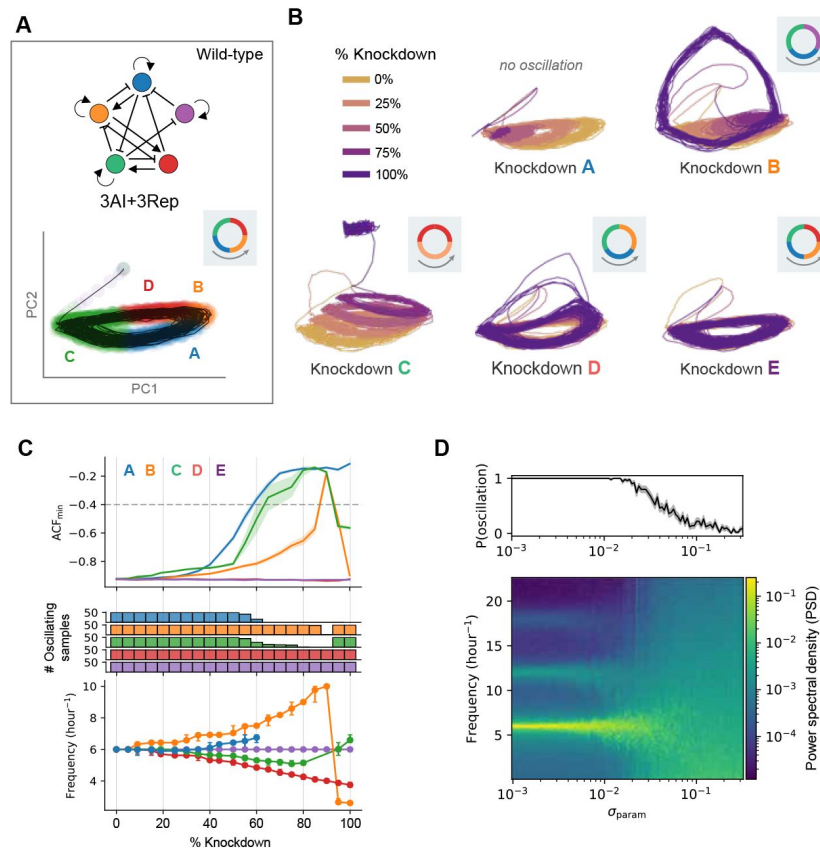


Figure 5.

Motif multiplexing makes oscillators resistant to the failure of components.

The 3AI+3Rep circuit (A, top) oscillates with different limit cycles after partial knockdowns of different genes. (A, bottom) An exemplary trajectory of 3AI+3Rep is shown on axes of the first two principal components (PCs) of phase space. Transparent circles indicate the dominant species at each time-point. (B) Trajectories under scenarios where transcription rate is reduced by a factor KD. The ordering of species in the limit cycle at KD = 100% is shown by the inset diagram. (C) Oscillation quality and frequency in single-gene knockdowns. Oscillations persist ($ACF_{\min} < ACF_{\text{thresh}}$) for most knockdowns of genes B, C, D, and E (middle). Oscillation frequency (bottom) is pulled from its WT value in knockdowns of A, B, C, and D. (E) Robustness to parameter variation between TFs. The power spectral density of the trajectory of TF A (bottom, mean) and the overall oscillation rate (top, mean $\pm$ SEM) are shown for simulations in which parameters were perturbed by a Gaussian kernel of width σ_{param} ($n = 50$ replicates). A dissipation of fundamental and harmonic frequencies and corresponding loss of oscillations occurs for $\sigma_{\text{param}} > 5 \cdot 10^{-2}$.

Discussion

Natural biological networks contain a number of biochemical components that exceeds our current capabilities of engineering by orders of magnitude, underscoring the importance of scalable computational methods for synthetic circuit design and analysis of biological design principles. Inspired by state-of-the-art artificial intelligence, we approach circuit design as a game of step-by-step topology assembly where success is determined by the achievement of a target phenotype in simulation. Similar to game-playing platforms that have achieved superhuman mastery of complex games (Silver et al. (2017) [link](#)), CircuiTree searches the space of possible circuit topologies using Monte Carlo tree search (MCTS), which balances exploitation of promising circuit assembly moves with exploration of other possibilities (Figure 1 [link](#)). This search strategy is comprehensive enough to infer motifs for a given phenotype (Figure 3 [link](#)) and efficient enough to search large spaces of designs fruitfully with limited samples (Figures 4A [link](#) and 4B [link](#)). Finally, we demonstrate CircuiTree's scalability by characterizing a novel class of five-gene oscillators that use a strategy we call motif multiplexing to resist deletion and single-gene knockdown mutations by densely interleaving multiple sub-oscillators (Figures 4C-4E [link](#) and 5A-5C [link](#)). This design principle may grant evolutionary robustness to eukaryotic circadian clocks, which have been observed across many phyla to contain multiple oscillatory loops (Cheng et al. (2001) [link](#); Bell-Pedersen et al. (2005) [link](#); Pokhilko et al. (2012) [link](#)), and it leaves open the question of how multiple sub-oscillators can be coupled without triggering chaos (Heltberg et al. (2021) [link](#)).

CircuiTree is distinguished by its general framework. Developed for general game-playing and planning problems, MCTS can query very large spaces in search of robust topologies and assembly motifs for any measurable phenotype, without restrictions on the modeling or simulation framework, and without needing to enumerate all possible topologies. This property bridges a gap in computational circuit design where generally, methods for finding single topologies (such as evolution (François and Hakim (2004) [link](#); François and Siggia (2008) [link](#)), mixed-integer optimization (Otero-Muras and Banga (2016) [link](#)), and recurrent neural networks (Shen et al. (2021) [link](#))) do not generalize easily to design principles and/or require a specific mathematical form, while more comprehensive methods (such as enumeration (Chau et al. (2012) [link](#); Schaerli et al. (2014) [link](#)) and Bayesian sampling (Woods et al. (2016) [link](#))) require an explicit list of topologies. Notably, while CircuiTree currently implements a “vanilla” version of the algorithm, MCTS can be modified in many ways to suit different problems (Browne et al. (2012) [link](#); Świechowski. (2022) [link](#)).

This work also highlights the possibilities of using reinforcement learning to address difficult combinatorial design and inference problems in biology. In future work, CircuiTree could be applied to combinatorial design of therapeutics, both in the sense of optimizing chimeric receptor design (Daniels et al. (2022) [link](#)) or combinatorial antigen recognition (Dannenfelser et al. (2020) [link](#); Williams et al. (2020) [link](#)), and in the sense of generating novel multicellular therapies consisting of multiple engineered cell types that collaborate *in situ*, analogous to a natural immune system. Our work could also be used to generate synthetic morphogenesis circuits, which are difficult to design due to the many possible combinations of chemical and physical components (Davies (2017) [link](#); Toda et al. (2018) [link](#); Fleischer and Barr (1997) [link](#)) and a high computational cost per simulation. Given a reward function that measures goodness-of-fit, CircuiTree could also be extended as an inference tool — for instance, for inferring transcriptional regulatory networks from data. More generally, this computational platform extends the study of design principles (Lim et al. (2013) [link](#)), which has been limited to small motifs of 2 or 3 components, to a larger space of biological networks, presenting an opportunity to dissect the algorithms and strategies used by biology to assemble complex networks at scale.

Methods

Modeling and simulation

Transcription factor (TF) cooperativity was modeled as a slower unbinding rate when both response elements (REs) for the a promoter are bound by the same TF. For K genes (each with mRNA and protein species) with cumulative A activation reactions and R repression reactions, the elimination of explicit dimerization reduces the number of species from $2K + K^2 + K^3$ (mRNAs, TFs, TF-TF complexes, and TF-TF-RE complexes) to $2K + A + R$ (mRNAs, TFs, and TF-RE complexes) and the number of reactions accordingly from $4K + 2K^2 + 3K^3$ to $4K + 3A + 3R$.

Stochastic trajectories were simulated using Gillespie's exact method (Gillespie (1977 [link](#))) and saved at $n_t = 2000$ time points at intervals of $dt = 20$ sec. The separation of time-scales between fast binding-unbinding kinetics and the other reactions creates a long simulation time (>2 mins) that cannot be alleviated with, for example, traditional τ -leaping (Cao et al. (2004 [link](#))). Thus, instead of using realistic but impractical binding and unbinding rates, these rates were set to virtual values determined from the equilibrium constant for first-binding $K_{D,1}$ by the solving the equations

$$K_{D,1} = k_{\text{off},1}/k_{\text{on}}$$

$$(k_{\text{off},1} \cdot 1 \text{ molec} + k_{\text{on}})(1 \text{ sec}) = Z,$$

where the value $Z = 100$ was chosen heuristically to be large enough to maintain a separation of time-scales at low quantities. For each protein species, high-frequency noise was filtered from the stochastic signal using a 9-point binomial filter (Aubury and Luk (1996 [link](#))), and the quality of oscillations overall was determined by computing the normalized autocorrelation function for each TF and finding the lowest minimum among TFs (ACF_{min}), excluding the bounds. Oscillations are considered present if this quantity, related to the dissipation constant for oscillations (Otero-Muras and Banga (2016 [link](#))), is below a cutoff $\text{ACF}_{\text{thresh}} = -0.4$.

For simulations with partial knockdown of a gene, all transcriptional rate parameters for that gene were multiplied by a coefficient on $[0, 1]$ (for example, KD=80% was achieved using a coefficient of 0.2.), and all species were initialized with zero quantity. For all other simulations, the initial quantity of each TF was selected from a Poisson distribution with a mean of 10 proteins, and all other species were initialized with zero quantity.

For perturbation studies, the eight sampled dimensionless variables were converted to values on $[0, 1]$ by normalization to the upper and lower values in **Table 2** [link](#). These values were stored in a $K \times 8$ matrix, where each row represented one of the $K = 5$ TFs, and each value was perturbed independently with a Gaussian kernel of standard deviation σ_{param} , truncated to the range $[0, 1]$ to prevent values outside the reasonable range. The perturbed values were then converted to rate parameters for each TF as outlined above.

Random sampling

Random sets of the 10 rate parameters listed in **Table 1** [link](#) were generated by drawing samples of the 8 dimensionless variables described in **Table 2** [link](#). Latin Hypercube sampling was used to draw 10^4 samples from a multivariate uniform distribution, with bounds shown in **Table 2** [link](#). For the 3-node and 5-node cases, every possible initialization (with a unique set of random generator seed, initial protein quantity, and parameters) was stored in a table of 10^4 rows, and each simulation was initialized with a random row of this table.

Monte Carlo tree search

For all MCTS runs, the hyperparameter in [Equation 2](#) was set to $c = 2.00$ to encourage exploration (the default value is $c = \sqrt{2}$). Replicate runs for an experiment were performed using different random seeds, which are used to break ties during the selection phase and draw a random parameter set during the simulation step. For the 3-node search, each reward value q was drawn as a Bernoulli trial with a success probability equal to the Q value found using enumeration rather than running fresh simulations.

MCTS was parallelized using the lock-free method [Enzenberger and Müller \(2010\)](#) and implemented with the Python utility ‘celery’ (<https://docs.celeryq.dev/en/stable/index.html>). Tree search was parallelized over 1,000 green threads which dispatched simulation jobs to run in parallel on multiple cloud computing instances totaling 300 CPUs. Reward values for each topology-parameter set pair were stored in an in-memory cache to speed up subsequent training runs (epochs). Back-propagation was executed asynchronously with virtual loss, assuming a reward of 0 until the actual reward is returned. To prevent excessive sampling of local optima during the 5-node search, we introduced a form of decision tree pruning we term “node exhaustion.” Once a terminal node (a completed topology) is visited $> 10^4$ times, it is considered exhaustively sampled and pruned from the search graph, and a non-terminal node is pruned once all its successors have been pruned. As illustrated in Figure S5, the visits and reward for each in-edge to an exhausted node are subtracted from the parent node. Thus, once a node is marked exhausted, its history is “forgotten” by its predecessors.

Please see the documentation at <https://pranav-bhamidipati.github.io/circuitree/index.html> for all additional details of implementation, as well as code tutorials and descriptions of the API.

Motif identification

Before testing for motifs based on a tree search, we first determine whether each terminated topology s_j discovered during the search is a successful oscillator by comparing its empirical robustness $\hat{Q}(s_j) = r_j/v_j$ to the predefined threshold $Q_{\text{thresh}} = 0.01$. This segregates all topologies into disjoint successful and unsuccessful sets (X and Y , respectively). We then take samples from the null distribution, drawing $n_{\text{sample}} = 10^5$ samples from the tree of topologies using random assembly, each time starting at the root of the design tree and choosing random actions until a terminal topology is reached (Figure S5A). The procedure is then repeated, this time rejecting the result unless it is a member of the successful set X (note that this can be computationally expensive if solutions are sparse). Next, a contingency table is constructed for each successful oscillator $x_i \in X$ to compare the frequency of observing x_i within samples of the successful subspace against the frequency of observation in the overall space. (see Figure S5A for an illustration of this process). Finally, statistically significant overrepresentation is determined by the χ^2 independence test with a significance threshold of $p < \alpha = 0.05$ after Bonferroni correction. To identify motifs based on the results of enumeration, we conduct the same hypothesis test on each oscillator, except using ground-truth frequencies found by enumeration.

Code availability

Circuitree, written in Python 3.10, is available on GitHub (<https://doi.org/10.5281/zenodo.11285522>). The code used to run computational experiments, perform analyses, and plot results is available separately at <https://doi.org/10.5281/zenodo.11285550>.

Acknowledgements

We thank M. Elowitz, J. Bois, A. Barr, L. Morsut, J. Courte, and J. Paulsson for scientific discussions. We thank J. Bois and G. Manela for advice and assistance on scientific software development. We thank J. Gornet for assistance with distributed computing. We are grateful to I.-M. Strazhnik for assistance with figures and illustrations.

Additional files

Supplementary Figures [↗](#)

References

- Alon U. (2007) **Network motifs: theory and experimental approaches** *Nature Reviews Genetics* Nature Publishing Group **8**:450–461 <https://doi.org/10.1038/nrg2102> | Google Scholar
- Alon U. (2019) **An Introduction to Systems Biology: Design Principles of Biological Circuits** Boca Raton: Chapman and Hall/CRC <https://doi.org/10.1201/9780429283321> | Google Scholar
- Aubury M, Luk W. (1996) **Binomial filters** *Journal of VLSI signal processing systems for signal, image and video technology* **12**:35–50 <https://doi.org/10.1007/BF00936945> | Google Scholar
- Auer P, Cesa-Bianchi N, Fischer P. (2002) **Finite-time Analysis of the Multiarmed Bandit Problem** *Machine Learning* **47**:235–256 <https://doi.org/10.1023/A:1013689704352> | Google Scholar
- Bell-Pedersen D, Cassone VM, Earnest DJ, Golden SS, Hardin PE, Thomas TL, Zoran MJ (2005) **Circadian rhythms from multiple oscillators: lessons from diverse organisms** *Nature Reviews Genetics* Nature Publishing Group **6**:544–556 <https://doi.org/10.1038/nrg1633> | Google Scholar
- Browne CB, Powley E, Whitehouse D, Lucas SM, Cowling PI, Rohlfshagen P, Tavener S, Perez D, Samothrakis S, Colton S. (2012) **A Survey of Monte Carlo Tree Search Methods** In: *IEEE Transactions on Computational Intelligence and AI in Games* pp. 1–43 <https://doi.org/10.1109/TCIAIG.2012.2186810> | Google Scholar
- Cao Y, Gillespie DT, Petzold LR (2004) **The slow-scale stochastic simulation algorithm** *The Journal of Chemical Physics* **122**:014116 <https://doi.org/10.1063/1.1824902> | Google Scholar
- Chandra FA, Buzi G, Doyle JC (2011) **Glycolytic Oscillations and Limits on Robust Efficiency** *Science* **333**:187–192 <https://doi.org/10.1126/science.1200705> | Google Scholar
- Chau AH, Walter JM, Gerardin J, Tang C, Lim WA (2012) **Designing Synthetic Regulatory Networks Capable of Self-Organizing Cell Polarization** *Cell* **151**:320–332 <https://doi.org/10.1016/j.cell.2012.08.040> | Google Scholar
- Cheng P, Yang Y, Liu Y. (2001) **Interlocked feedback loops contribute to the robustness of the Neurospora circadian clock** *Proceedings of the National Academy of Sciences* Proceedings of the National Academy of Sciences **98**:7408–7413 <https://doi.org/10.1073/pnas.121170298> | Google Scholar
- Cotterell J, Sharpe J. (2010) **An atlas of gene regulatory networks reveals multiple three-gene mechanisms for interpreting morphogen gradients** *Molecular Systems Biology* **6**:425 <https://doi.org/10.1038/msb.2010.74> | Google Scholar
- Daniels KG, Wang S, Simic MS, Bhargava HK, Capponi S, Tonai Y, Yu W, Bianco S, Lim WA (2022) **Decoding CAR T cell phenotype using combinatorial signaling motif libraries and machine learning** *Science* **378**:1194–1200 <https://doi.org/10.1126/science.abq0225> | Google Scholar

- Dannenfelser R, Allen GM, VanderSluis B, Koegel AK, Levinson S, Stark SR, Yao V, Tadych A, Troyanskaya OG, Lim WA (2020) **Discriminatory Power of Combinatorial Antigen Recognition in Cancer T Cell Therapies** *Cell Systems* **11**:215–228 <https://doi.org/10.1016/j.cels.2020.08.002> | Google Scholar
- Davies J. (2017) **Using synthetic biology to explore principles of development** *Development* **144**:1146–1158 <https://doi.org/10.1242/dev.144196> | Google Scholar
- Elowitz MB, Leibler S. (2000) **A synthetic oscillatory network of transcriptional regulators** *Nature* **403**:335–338 <https://doi.org/10.1038/35002125> | Google Scholar
- Enzenberger M, Müller M. (2010) **A Lock-Free Multithreaded Monte-Carlo Tree Search Algorithm** In: van den Herik HJ, Spronck P, editors. *Advances in Computer Games Lecture Notes in Computer Science* Berlin, Heidelberg: Springer pp. 14–20 https://doi.org/10.1007/978-3-642-12993-3_2 | Google Scholar
- Ferrell JE, Tsai TYC, Yang Q. (2011) **Modeling the Cell Cycle: Why Do Certain Circuits Oscillate?** *Cell* **144**:874–885 <https://doi.org/10.1016/j.cell.2011.03.006> | Google Scholar
- Fleischer K, Barr A. (1997) **A Simulation Testbed for the Study of Multicellular Development: The Multiple Mechanisms of Morphogenesis** *Artificial Life III* | Google Scholar
- François P, Hakim V. (2004) **Design of genetic networks with specified functions by evolution in silico** *Proceedings of the National Academy of Sciences* **101**:580–585 <https://doi.org/10.1073/pnas.0304532101> | Google Scholar
- François P, Hakim V, Siggia ED. (2007) **Deriving structure from evolution: metazoan segmentation** *Molecular Systems Biology* **3**:154 <https://doi.org/10.1038/msb4100192> | Google Scholar
- François P, Siggia ED. (2008) **A case study of evolutionary computation of biochemical adaptation** *Physical Biology* **5**:026009 <https://doi.org/10.1088/1478-3975/5/2/026009> | Google Scholar
- François P, Siggia ED. (2010) **Predicting embryonic patterning using mutual entropy fitness and in silico evolution** *Development* **137**:2385–2395 <https://doi.org/10.1242/dev.048033> | Google Scholar
- Gardner TS, Cantor CR, Collins JJ (2000) **Construction of a genetic toggle switch in Escherichia coli** *Nature* **403**:339–342 <https://doi.org/10.1038/35002131> | Google Scholar
- Gerardin J, Reddy NR, Lim WA (2019) **The Design Principles of Biochemical Timers: Circuits that Discriminate between Transient and Sustained Stimulation** *Cell Systems* **9**:297–308 <https://doi.org/10.1016/j.cels.2019.07.008> | Google Scholar
- Geva-Zatorsky N, Dekel E, Batchelor E, Lahav G, Alon U. (2010) **Fourier analysis and systems identification of the p53 feedback loop** *Proceedings of the National Academy of Sciences* **107**:13550–13555 <https://doi.org/10.1073/pnas.1001107107> | Google Scholar
- Geva-Zatorsky N, Rosenfeld N, Itzkovitz S, Milo R, Sigal A, Dekel E, Yarnitzky T, Liron Y, Polak P, Lahav G, Alon U. (2006) **Oscillations and variability in the p53 system** *Molecular Systems Biology* **2** <https://doi.org/10.1038/msb4100068> | Google Scholar

- Gillespie DT (1977) **Exact stochastic simulation of coupled chemical reactions** *The Journal of Physical Chemistry* **81**:2340–2361 <https://doi.org/10.1021/j100540a008> | Google Scholar
- Heltberg ML, Krishna S, Kadanoff LP, Jensen MH (2021) **A tale of two rhythms: Locked clocks and chaos in biology** *Cell Systems* **12**:291–303 <https://doi.org/10.1016/j.cels.2021.03.003> | Google Scholar
- Jiménez A, Cotterell J, Munteanu A, Sharpe J. (2017) **A spectrum of modularity in multi-functional gene circuits** *Molecular Systems Biology* **13**:925 <https://doi.org/10.15252/msb.20167347> | Google Scholar
- Jones EM, Marken JP, Silver PA (2024) **Synthetic microbiology in sustainability applications** *Nature Reviews Microbiology* :1–15 <https://doi.org/10.1038/s41579-023-01007-9> | Google Scholar
- Kocsis L, Szepesvári C. (2006) **Bandit Based Monte-Carlo Planning** In: Fürnkranz J, Scheffer T, Spiliopoulou M, editors. *Machine Learning: ECML 2006 Lecture Notes in Computer Science* Berlin, Heidelberg: Springer pp. 282–293 https://doi.org/10.1007/11871842_29 | Google Scholar
- Lee K, Loros JJ, Dunlap JC (2000) **Interconnected Feedback Loops in the Neurospora Circadian System** *Science* **289**:107–110 <https://doi.org/10.1126/science.289.5476.107> | Google Scholar
- Lim WA, Lee CM, Tang C. (2013) **Design Principles of Regulatory Networks: Searching for the Molecular Algorithms of the Cell** *Molecular Cell* **49**:202–212 <https://doi.org/10.1016/j.molcel.2012.12.020> | Google Scholar
- Ma W, Trusina A, El-Samad H, Lim WA, Tang C. (2009) **Defining Network Topologies that Can Achieve Biochemical Adaptation** *Cell* **138**:760–773 <https://doi.org/10.1016/j.cell.2009.06.013> | Google Scholar
- Milo R, Shen-Orr S, Itzkovitz S, Kashtan N, Chklovskii D, Alon U. (2002) **Network Motifs: Simple Building Blocks of Complex Networks** *Science* **298**:824–827 <https://doi.org/10.1126/science.298.5594.824> | Google Scholar
- Milo R, Jorgensen P, Moran U, Weber G, Springer M. (2010) **BioNumbers—the database of key numbers in molecular and cell biology** *Nucleic Acids Research* **38**:D750–D753 <https://doi.org/10.1093/nar/gkp889> | Google Scholar
- Novák B, Tyson JJ (2008) **Design principles of biochemical oscillators** *Nature Reviews Molecular Cell Biology* **9**:981–991 <https://doi.org/10.1038/nrm2530> | Google Scholar
- Otero-Muras I, Banga JR (2016) **Design Principles of Biological Oscillators through Optimization: Forward and Reverse Analysis** *PLOS ONE* **11**:e0166867 <https://doi.org/10.1371/journal.pone.0166867> | Google Scholar
- Pokhilko A, Fernández AP, Edwards KD, Southern MM, Halliday KJ, Millar AJ (2012) **The clock gene circuit in Arabidopsis includes a repressilator with additional feedback loops** *Molecular Systems Biology* **8**:574 <https://doi.org/10.1038/msb.2012.6> | Google Scholar

- Purnick PEM, Weiss R. (2009) **The second wave of synthetic biology: from modules to systems** *Nature Reviews Molecular Cell Biology* **10**:410–422 <https://doi.org/10.1038/nrm2698> | [Google Scholar](#)
- Qiao L, Zhang ZB, Zhao W, Wei P, Zhang L. (2022) **Network design principle for robust oscillatory behaviors with respect to biological noise** *eLife* **11**:e76188 <https://doi.org/10.7554/eLife.76188> | [Google Scholar](#)
- Qiao L, Zhao W, Tang C, Nie Q, Zhang L. (2019) **Network Topologies That Can Achieve Dual Function of Adaptation and Noise Attenuation** *Cell Systems* **9**:271–285 <https://doi.org/10.1016/j.cels.2019.08.006> | [Google Scholar](#)
- Schaerli Y, Munteanu A, Gili M, Cotterell J, Sharpe J, Isalan M. (2014) **A unified design space of synthetic stripe-forming networks** *Nature Communications* **5**:4905 <https://doi.org/10.1038/ncomms5905> | [Google Scholar](#)
- Shah NA, Sarkar CA (2011) **Robust Network Topologies for Generating Switch-Like Cellular Responses** *PLOS Computational Biology* **7**:e1002085 <https://doi.org/10.1371/journal.pcbi.1002085> | [Google Scholar](#)
- Shannon C. (1950) **Programming a Computer for Playing Chess** *Philosophical Magazine* **41** <https://www.computerhistory.org/chess/doc-431614f453dde/> | [Google Scholar](#)
- Shen J, Liu F, Tu Y, Tang C. (2021) **Finding gene network topologies for given biological function with recurrent neural network** *Nature Communications* **12**:3125 <https://doi.org/10.1038/s41467-021-23420-5> | [Google Scholar](#)
- Silver D, Huang A, Maddison CJ, Guez A, Sifre L, van den Driessche G, Schrittwieser J, Antonoglou I, Panneershelvam V, Lanctot M, Dieleman S, Grewe D, Nham J, Kalchbrenner N, Sutskever I, Lillicrap T, Leach M, Kavukcuoglu K, Graepel T, Hassabis D. (2016) **Mastering the game of Go with deep neural networks and tree search** *Nature* **529**:484–489 <https://doi.org/10.1038/nature16961> | [Google Scholar](#)
- Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, Lanctot M, Sifre L, Kumaran D, Graepel T, Lillicrap T, Simonyan K, Hassabis D. (2018) **A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play** *Science* **362**:1140–1144 <https://doi.org/10.1126/science.aar6404> | [Google Scholar](#)
- Silver D, Schrittwieser J, Simonyan K, Antonoglou I, Huang A, Guez A, Hubert T, Baker L, Lai M, Bolton A, Chen Y, Lillicrap T, Hui F, Sifre L, van den Driessche G, Graepel T, Hassabis D. (2017) **Mastering the game of Go without human knowledge** *Nature* **550**:354–359 <https://doi.org/10.1038/nature24270> | [Google Scholar](#)
- Stricker J, Cookson S, Bennett MR, Mather WH, Tsimring LS, Hasty J. (2008) **A fast, robust and tunable synthetic gene oscillator** *Nature* **456**:516–519 <https://doi.org/10.1038/nature07389> | [Google Scholar](#)
- Toda S, Blauch LR, Tang SKY, Morsut L, Lim WA (2018) **Programming self-organizing multicellular structures with synthetic cell-cell signaling** *Science* **361**:156–162 <https://doi.org/10.1126/science.aat0271> | [Google Scholar](#)

- Tromp J, Farneback G. (2007) **Combinatorics of Go** In: *Computers and Games Lecture Notes in Computer Science* pp. 84–99 https://doi.org/10.1007/978-3-540-75538-8_8 | Google Scholar
- Tyson JJ, Hong CI, Dennis Thron C, Novak B. (1999) **A Simple Model of Circadian Rhythms Based on Dimerization and Proteolysis of PER and TIM** *Biophysical Journal* **77**:2411–2417 [https://doi.org/10.1016/S0006-3495\(99\)77078-5](https://doi.org/10.1016/S0006-3495(99)77078-5) | Google Scholar
- Wagner A. (2005) **Circuit topology and the evolution of robustness in two-gene circadian oscillators** *Proceedings of the National Academy of Sciences* **102**:11775–11780 <https://doi.org/10.1073/pnas.0501094102> | Google Scholar
- Williams JZ, Allen GM, Shah D, Sterin IS, Kim KH, Garcia VP, Shavey GE, Yu W, Puig-Saus C, Tsoi J, Ribas A, Roybal KT, Lim WA (2020) **Precise T cell recognition programs designed by transcriptionally linking multiple receptors** *Science* **370**:1099–1104 <https://doi.org/10.1126/science.abc6270> | Google Scholar
- Woods ML, Leon M, Perez-Carrasco R, Barnes CP (2016) **A Statistical Approach Reveals Designs for the Most Robust Stochastic Gene Oscillators** *ACS Synthetic Biology* **5**:459–470 <https://doi.org/10.1021/acssynbio.5b00179> | Google Scholar
- Zhu R, del Rio-Salgado JM, Garcia-Ojalvo J, Elowitz MB (2022) **Synthetic multistability in mammalian cells** *Science* **375**:eabg9765 <https://doi.org/10.1126/science.abg9765> | Google Scholar
- świechowski M, Godlewski K, Sawicki B, Mańdziuk J. (2022) **Monte Carlo Tree Search: a review of recent modifications and applications** *Artificial Intelligence Review* <https://doi.org/10.1007/s10462-022-10228-y> | Google Scholar

Author information

Pranav S Bhamidipati

Division of Biology and Biological Engineering, California Institute of Technology, Pasadena, United States, Keck School of Medicine, University of Southern California, Los Angeles, United States

ORCID iD: [0000-0002-6199-6505](https://orcid.org/0000-0002-6199-6505)

For correspondence: pbhamidi@usc.edu

Matthew Thomson

Division of Biology and Biological Engineering, California Institute of Technology, Pasadena, United States, Department of Computing and Mathematical Sciences, California Institute of Technology, Pasadena, United States, Beckman Center for Single-cell Profiling and Engineering, Pasadena, United States

ORCID iD: [0000-0003-1021-1234](https://orcid.org/0000-0003-1021-1234)

For correspondence: mthomson@caltech.edu

Editors

Reviewing Editor

Mariana Gómez-Schiavon

Universidad Nacional Autónoma de México, Querétaro, Mexico

Senior Editor

Aleksandra Walczak

CNRS, Paris, France

Joint Public Review:

This manuscript presents an algorithm for identifying network topologies that exhibit a desired qualitative behaviour, with a particular focus on oscillations. The approach is first demonstrated on 3-node networks, where results can be validated through exhaustive search, and then extended to 5-node networks, where the search space becomes intractable. Network topologies are represented as directed graphs, and their dynamical behaviour is classified using stochastic simulations based on the Gillespie algorithm. To efficiently explore the large design space, the authors employ reinforcement learning via Monte Carlo Tree Search (MCTS), framing circuit design as a sequential decision-making process.

This work meaningfully extends the range of systems that can be explored *in silico* to uncover non-linear dynamics and represents a valuable methodological advance for the fields of systems and synthetic biology.

Strengths

The evidence presented is strong and compelling. The authors validate their results for 3-node networks through exhaustive search, and the findings for 5-node networks are consistent with previously reported motifs, lending credibility to the approach. The use of reinforcement learning to navigate the vast space of possible topologies is both original and effective, and represents a novel contribution to the field. The algorithm demonstrates convincing efficiency, and the ability to identify robust oscillatory topologies is particularly valuable. Expanding the scale of systems that can be systematically explored *in silico* marks a significant advance for the study of complex gene regulatory networks.

Weaknesses

The principal weakness of the manuscript lies in the interpretation of biological robustness. The authors identify network topologies that sustain oscillatory behaviour despite perturbations to the system or parameters. However, in many cases, this persistence is due to the presence of partially redundant oscillatory motifs within the network. While this observation is interesting and of clear value for circuit design, framing it as evidence of evolutionary robustness may be misleading. The "mutant" systems frequently exhibit altered oscillatory properties, such as changes in frequency or amplitude. From a functional cellular perspective, mere oscillation is insufficient - preservation of specific oscillation characteristics is often essential. This is particularly true in systems like circadian clocks, where misalignment with environmental cycles can have deleterious effects. Robustness, from an evolutionary standpoint, should therefore be framed as the capacity to maintain the functional phenotype, not merely the qualitative behaviour.

A secondary limitation is that, despite the methodological advances, the scale of the systems explored remains modest. While moving from 3- to 5-node systems is non-trivial, five elements still represent a relatively small network. It is somewhat surprising that the algorithm does not scale further, particularly when considering the performance of MCTS in

other domains - for instance, modern chess engines routinely explore far larger decision trees. A discussion on current performance bottlenecks and potential avenues for improving scalability would be valuable.

Finally, it is worth noting that the emergence of oscillations in a model often depends not only on the topology but also critically on parameter choices and the nature of the nonlinearities. The use of Hill functions and high Hill coefficients is a common strategy to induce oscillatory dynamics. Thus, the reported results should be interpreted within the context of the modelling assumptions and parameter regimes employed in the simulations.

<https://doi.org/10.7554/eLife.106497.1.sa0>

Author response:

Public Review

Joint Public Review:

This manuscript presents an algorithm for identifying network topologies that exhibit a desired qualitative behaviour, with a particular focus on oscillations. The approach is first demonstrated on 3-node networks, where results can be validated through exhaustive search, and then extended to 5-node networks, where the search space becomes intractable. Network topologies are represented as directed graphs, and their dynamical behaviour is classified using stochastic simulations based on the Gillespie algorithm. To efficiently explore the large design space, the authors employ reinforcement learning via Monte Carlo Tree Search (MCTS), framing circuit design as a sequential decision-making process.

This work meaningfully extends the range of systems that can be explored in silico to uncover non-linear dynamics and represents a valuable methodological advance for the fields of systems and synthetic biology.

Strengths

The evidence presented is strong and compelling. The authors validate their results for 3-node networks through exhaustive search, and the findings for 5-node networks are consistent with previously reported motifs, lending credibility to the approach. The use of reinforcement learning to navigate the vast space of possible topologies is both original and effective, and represents a novel contribution to the field. The algorithm demonstrates convincing efficiency, and the ability to identify robust oscillatory topologies is particularly valuable. Expanding the scale of systems that can be systematically explored in silico marks a significant advance for the study of complex gene regulatory networks.

Weaknesses

The principal weakness of the manuscript lies in the interpretation of biological robustness. The authors identify network topologies that sustain oscillatory behaviour despite perturbations to the system or parameters. However, in many cases, this persistence is due to the presence of partially redundant oscillatory motifs within the network. While this observation is interesting and of clear value for circuit design, framing it as evidence of evolutionary robustness may be misleading. The "mutant" systems frequently exhibit altered oscillatory properties, such as changes in frequency or amplitude. From a functional cellular perspective, mere oscillation is insufficient - preservation of specific oscillation characteristics is often essential. This is particularly true in systems like circadian clocks, where misalignment with environmental cycles can

have deleterious effects. Robustness, from an evolutionary standpoint, should therefore be framed as the capacity to maintain the functional phenotype, not merely the qualitative behaviour.

A secondary limitation is that, despite the methodological advances, the scale of the systems explored remains modest. While moving from 3- to 5-node systems is non-trivial, five elements still represent a relatively small network. It is somewhat surprising that the algorithm does not scale further, particularly when considering the performance of MCTS in other domains - for instance, modern chess engines routinely explore far larger decision trees. A discussion on current performance bottlenecks and potential avenues for improving scalability would be valuable.

Finally, it is worth noting that the emergence of oscillations in a model often depends not only on the topology but also critically on parameter choices and the nature of the nonlinearities. The use of Hill functions and high Hill coefficients is a common strategy to induce oscillatory dynamics. Thus, the reported results should be interpreted within the context of the modelling assumptions and parameter regimes employed in the simulations.

We thank the reviewers for their careful consideration of our work and for the interesting feedback and scientific discussion. We are working on a revised text based on their recommendations, which will include some of the discussion below.

This work meaningfully extends the range of systems that can be explored in silico to uncover non-linear dynamics and represents a valuable methodological advance for the fields of systems and synthetic biology.

We thank the reviewers for their positive assessment of our work's impact!

The use of reinforcement learning to navigate the vast space of possible topologies is both original and effective, and represents a novel contribution to the field. The algorithm demonstrates convincing efficiency, and the ability to identify robust oscillatory topologies is particularly valuable. Expanding the scale of systems that can be systematically explored in silico marks a significant advance for the study of complex gene regulatory networks.

We appreciate these kind comments about our work's merits. We are excited to share our reinforcement learning (RL) based method with the fields of systems and synthetic biology, and we consider it a valuable tool for the systematic analysis and design of larger-scale regulatory networks!

The principal weakness of the manuscript lies in the interpretation of biological robustness. The authors identify network topologies that sustain oscillatory behaviour despite perturbations to the system or parameters... [However, these] "mutant" systems frequently exhibit altered oscillatory properties, such as changes in frequency or amplitude. From a functional cellular perspective, mere oscillation is insufficient - preservation of specific oscillation characteristics is often essential. This is particularly true in systems like circadian clocks, where misalignment with environmental cycles can have deleterious effects. Robustness, from an evolutionary standpoint, should therefore be framed as the capacity to maintain the functional phenotype, not merely the qualitative behaviour.

We thank the reviewers for their attention to this point. In the large-scale circuit search, summarized in Figures 4A and 4B, we ran a search for 5-component oscillators that can spontaneously oscillate even when subjected to the deletion of a random gene. Some of the

best performing circuits under these conditions exhibited a design feature we call “motif multiplexing,” in which multiple smaller motifs are interleaved in a way that makes oscillation possible under many different mutational scenarios. Interestingly, despite not selecting for preservation of frequency, the 3Ai+3Rep circuit (a 5-gene circuit highlighted in Figure 5) anecdotally appears to have a natural frequency that is robust to partial gene knockdowns, although not to complete gene deletions. As shown in Figure 5C, this circuit has a natural frequency of 6 cycles/hr (with one particular parameterization), and it can sustain a knockdown of any of its 5 genes to 50% of the wild-type transcription rate without altering the natural frequency by more than 20%.

However, we agree that there are salient differences between this training scenario and natural evolution. The revised text will clarify that these differences limit what conclusions can be drawn about biological evolution by analogy. As the reviewers point out, we use the presence of spontaneous oscillations (with or without the deletion) as a measure of fitness, regardless of frequency, so as to screen for designs with promising behavior. Also, the deletion mutations introduced during training likely represent larger perturbations to the system than a typical mutation encountered during genome replication (for example, a point mutation in a response element leading to a moderate change in binding affinity). Finally, we do not introduce any entrainment. Real circadian oscillators are aligned to a 24-hour period (“entrained”) by environmental inputs such as light and temperature. For this reason, natural circadian clocks may have natural frequencies that are slightly shorter or longer than 24 hours, although a close proximity to the 24-hour period does seem to be an important selective factor [1].

...despite the methodological advances, the scale of the systems explored remains modest. While moving from 3- to 5-node systems is non-trivial, five elements still represent a relatively small network. It is somewhat surprising that the algorithm does not scale further, particularly when considering the performance of MCTS in other domains - for instance, modern chess engines routinely explore far larger decision trees. A discussion on current performance bottlenecks and potential avenues for improving scalability would be valuable.

We thank the reviewers for their attention to this point. The main limitation we encountered to exploring circuits with more than 5 nodes in this work was the poor computational scaling of the Gillespie stochastic simulation algorithm, rather than a limitation of MCTS itself. While the average runtime of a 3-node circuit simulation was roughly 7 seconds, this number increased to 18-20 seconds with 5-node circuits. For this reason, we limited the search to topologies with ≤ 15 interaction arrows (15 sec/simulation). In general, the simulation time was proportional to the square of the number of transcription factors (TFs). We will revise the text to include the reason for stopping at 5 nodes, which is significant for understanding CircuiTree’s scaling properties.

With regards to scaling, an important advantage of CircuiTree is its ability to generate useful candidate designs after exploring only a portion of the search space. Like exhaustive search, given enough time, MCTS will comprehensively explore the search space and find all possible solutions. However, for large search spaces, RL-based agents are generally given a finite number of simulations (or time) to learn as much as possible.

Across machine learning (ML) applications [2] and particularly with RL models [3], this training time tends to obey a power law with respect to the underlying complexity of the problem. Thus we can use the complexity of the 3-node and 5-node searches to infer the current scaling limits of CircuiTree. The first oscillator topology was discovered after 2,280 simulations for the 3-node search, and in the 5-node search, the first oscillator using 5 nodes appeared at $\sim 8e5$ simulations, resulting in a power law of $Y \sim 84.4 X^{0.333}$. Thus, useful candidate designs may be found for 6-node and 7-node searches after $4.5e7$ and $5.26e9$

simulations, respectively, even though these spaces contain 1.5×10^{17} and 2.5×10^{23} topologies, respectively. Thus, running a 7-node search with the current implementation of CircuiTree would require resources close to the current boundaries of computation, requiring roughly 1.8 million CPU-hours, or 2 weeks on 5,000 CPUs, assuming a 1-second simulation. These points will be incorporated into both the results and discussion sections in our revised text.

However, we are optimistic about CircuiTree's potential to scale to much larger circuits with modifications to its algorithm. CircuiTree uses the original (so-called “vanilla”) implementation of MCTS, which has not been used in professional game-playing AIs in over a decade. Contemporary RL-based game-playing engines leverage deep neural networks to dramatically reduce the training time, using value networks to identify game-winning positions and policy networks to find game-winning moves. AlphaZero, developed by Google DeepMind to learn games by self-play and without domain knowledge, outperformed all other chess AIs after 44 million training games, much smaller than the 10^{43} possible chess states [4]. Similarly, the game of go has 10^{170} possible states, but AlphaZero outperformed other AIs after only 140 million games [4]. Large circuits live in similarly large search spaces; for example, 19-node and 20-node circuits represent spaces of 10^{172} and 10^{190} possible topologies. The revised text will include this discussion and identify value and policy networks, as well as more scalable simulation paradigms such as ODEs and neural ODEs, as our future directions for improving CircuiTree's scalability.

Finally, our revised discussion will note some important differences between game-playing and biological circuit design. Unlike deterministic games like chess, the final value of a circuit topology is determined stochastically, by running a simulation whose fitness depends on the parameter set and initial conditions. Thus, state-for-state, it is possible that training an agent for circuit design may inherently require more simulations to achieve the same level of certainty compared to classical games. Additionally, while we often possess a priori knowledge about a game such as its overall difficulty or certain known strategies, we lack this frame of reference when searching for circuit designs. Thus, it remains challenging to know if and when a large space of designs has been “satisfactorily” or “comprehensively” searched, since the answer depends on data that are unknown, namely the quantity, quality, and location of solutions residing in the search space.

Not accounting for redundancy due to structural symmetries

Finally, it is worth noting that the emergence of oscillations in a model often depends not only on the topology but also critically on parameter choices and the nature of the nonlinearities. The use of Hill functions and high Hill coefficients is a common strategy to induce oscillatory dynamics. Thus, the reported results should be interpreted within the context of the modelling assumptions and parameter regimes employed in the simulations.

In our dynamical modeling of transcription factor (TF) networks, we do not rely on continuum assumptions about promoter occupancy such as Hill functions. Rather, we model each reaction - transcription, translation, TF binding/unbinding, and degradation - explicitly, and individual molecules appear and disappear via stochastic birth and death events. Many natural TFs are homodimers that bind cooperatively to regulate transcription; similarly, we assume that pairs of TFs bind more stably to their response element than individual TFs. Thus, our model has similar cooperativity to a Hill function, and it can be shown that in the continuum limit, the effective Hill coefficient is always ≤ 2 . Our revision will clarify this aspect of the modeling and include a derivation of this property. Currently, the parameter values used in the figures are shown in Table 2. In the revised text, these will be displayed in the body of the text as well for clarity.

Bibliography (1) Spoelstra, K., Wikelski, M., Daan, S., Loudon, A. S. I., & Hau, M. (2015). Natural selection against a circadian clock gene mutation in mice. *PNAS*, 113(3), 686–691. <https://doi.org/10.1073/pnas.1516442113>
(2) Neumann, O., & Gros, C. (2023). Scaling Laws for a Multi-Agent Reinforcement Learning Model. The Eleventh International Conference on Learning Representations. Retrieved from <https://openreview.net/forum?id=ZrEbzL9eQ3W> (3) Jones, A. L. (2021). Scaling Scaling Laws with Board Games. arXiv [Cs.LG]. Retrieved from <http://arxiv.org/abs/2104.03113> (4) Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that Masters Chess, Shogi, and go through self-play. *Science*, 362(6419), 1140–1144. <https://doi.org/10.1126/science.aar6404>

<https://doi.org/10.7554/eLife.106497.1.sa2>